University of Genoa

Machine Learning Project

StarCraft II Match Outcome Prediction

From Replay Parsing to Group-Aware Experimental Validation

This document is designed as the **main technical paper** of the project. It consolidates the project methodology, explains the logic of the pipeline, and reports the final replay-aware experimental results preserved in the repository for reproducibility review.

Candidate: Pineda Fritas Jose Luis
Academic year: 2025–2026
Document version: Final revised version
Date: March 2026

Abstract

This project studies match outcome prediction in *StarCraft II* using tabular snapshots extracted from real replay files rather than images or raw sequences. The technical objective is not only to train a classifier, but to build a complete and reproducible pipeline from replay parsing to inference. The dataset is generated from more than 24,000 competitive replays and sampled every 15 seconds after removing the first two minutes of game time. The work focuses on domain-aware feature engineering, group-aware evaluation to avoid replay leakage, and comparative analysis across Random Forest, XGBoost, and deep multilayer perceptrons.

A central methodological result is that naive row-based validation substantially overestimates performance because adjacent snapshots from the same replay are highly correlated. Once the split is performed by `replay_id`, the final evaluation settles near 61%, which is far more realistic and scientifically meaningful. Tree-based models then emerge as the strongest and most stable baseline on this tabular representation. The final GPU-trained MLP remains competitive but does not surpass the tree ensembles, and the strongest remaining advantage of XGBoost over Random Forest appears in probabilistic quality rather than in a large gain in raw accuracy. Overall, the results support replay-aware tree ensembles as the strongest and most reliable baseline for this tabular representation of StarCraft II match state.

Contents

1 Introduction and Project Goal **2**

2 Problem Formulation **2**

3 Pipeline Overview **2**

4 Data Acquisition and Dataset Construction **3**

4.1 Replay source and scale 3

4.2 Sampling policy 3

4.3 Why the first 120 seconds are removed 3

4.4 Dataset identity structure 3

5 Feature Engineering Logic **3**

5.1 Feature families 3

5.2 Combat score 4

5.3 Economic features 4

5.4 Scouting and composition features 4

5.5 Dynamic features 4

6 Methodological Pivot: Leakage Control **5**

7 Model Families and Expected Behavior **5**
7.1 Random Forest . . . 5
7.2 XGBoost . . . 5
7.3 MLP . . . 5
7.4 Why trees may be favored here . . . 5

8 Experimental Protocol **6**
8.1 Baseline data preparation . . . 6
8.2 Random Forest protocol . . . 6
8.3 XGBoost protocol . . . 6
8.4 MLP protocol status . . . 6

9 Current Results Snapshot **7**
9.1 Consolidated ablation status up to the current iteration . . . 7
9.2 Current best XGBoost reference run . . . 8
9.3 Random Forest cross-check on the fixed dataset . . . 8
9.4 Final cross-model comparison on the frozen main dataset . . . 9
9.5 Paired replay-bootstrap significance test: XGBoost vs Random Forest . . . 9
9.6 Calibration analysis of the final tree ensembles . . . 10
9.7 Calibration completion with the final MLP export . . . 11
9.8 Frozen ablation and reproducibility table . . . 12

10 Feature Importance and Interpretation **12**

11 Temporal Analysis by Match Duration **15**

12 Critical Discussion **17**
12.1 Why the absolute accuracy is not extremely high . . . 17
12.2 What the models are probably learning . . . 17
12.3 Why trees currently dominate . . . 17

13 Threats to Validity **18**

14 Conclusion **18**

15 Conceptual Extension: A POMDP-like View of Replay Snapshots **18**

A Appendix A: Implementation Snapshot **19**

B Appendix B: Supplementary Note on the POMDP-like Interpretation 19

1 Introduction and Project Goal

StarCraft II is a real-time strategy game with hidden information, long-horizon planning, macroeconomic management, and highly nonlinear interactions between army composition, technology, scouting, and timing. Predicting the winner from replay-derived tabular data is therefore an interesting machine learning problem for two reasons. First, it is a difficult supervised task with a meaningful target. Second, it forces the project to solve the much harder problem that comes *before* training: how to convert a noisy temporal game trace into a structured dataset that preserves strategic signal.

The project goal is therefore twofold:

1. build a replay-to-table pipeline that extracts strategically meaningful features at fixed time intervals;
2. evaluate multiple supervised learning paradigms under a protocol that avoids information leakage between snapshots from the same match.

The classifier is only the last stage of the pipeline. The scientific value of the project depends equally on parser design, feature semantics, data cleaning, and validation protocol.

2 Problem Formulation

Let each replay generate a sequence of snapshots x_t sampled at time t, with label $y \in \{0, 1\}$ indicating whether Player 1 wins the match. The learning objective is to approximate

$$f(x_t) \approx P(y = 1 \mid x_t)$$

using a tabular representation of the game state. This framing has an immediate limitation: the model does not observe the full spatial or sequential structure of the game. Instead, it only sees the information that the parser chooses to encode. For this reason, the feature space must capture not only absolute state variables, but also relationships between the two players and local temporal momentum.

3 Pipeline Overview

The project pipeline can be summarized as follows:

$$\begin{gathered}\text{Replay corpus} \to \text{SC2 parser / bridge} \to \text{snapshot dataset} \\ \to \text{group-aware split} \to \text{model selection} \to \text{final evaluation}\end{gathered}$$

In practical terms, the project contains three tightly coupled layers:

- **Replay ingestion**: reading `.SC2Replay` files and recovering player-level state at multiple times;
- **Feature engineering**: compressing military, economic, scouting, and compositional information into a fixed-width row;
- **Learning and validation**: comparing model families under a replay-aware evaluation protocol.

4 Data Acquisition and Dataset Construction

4.1 Replay source and scale

The broader project material references roughly **24,057** raw competitive or high-level replays, while the frozen **V3.1 fixed** benchmark retained **23,241** usable replays and **892,047** snapshots after the final filtering and parser-stability corrections. The extraction pipeline uses `sc2reader` for replay parsing and `joblib` for parallel execution. A key engineering point is that replay parsing is heavy enough that preprocessing itself becomes a relevant computational stage.

4.2 Sampling policy

Snapshots are extracted every **15 seconds**. Extremely short or atypically long games are removed, with the current filtering policy centered on durations between **180 s** and **1800 s**. In addition, the first **120 seconds** are removed from the learning problem because the opening phase is comparatively deterministic and contributes less discriminative variance.

4.3 Why the first 120 seconds are removed

The early game in StarCraft II often follows race-specific build order routines with low observable divergence in the aggregate statistics used here. Removing the earliest phase acts as a denoising step: it reduces the proportion of nearly identical low-information states and lets the models focus on strategic separation that emerges after the opening.

4.4 Dataset identity structure

Each row is associated with at least:

- a `replay_id`,
- a snapshot timestamp `time_sec`,
- a binary label `p1_wins`.

This identity information is not just metadata. It is essential for correct validation because all snapshots belonging to the same replay are statistically dependent.

5 Feature Engineering Logic

5.1 Feature families

The current project design includes four main semantic families:

1. **Military power and army state**
2. **Economic efficiency and production balance**
3. **Scouting and informational behavior**
4. **Temporal dynamics and momentum**

The earlier presentation material reports around **30 columns** in the baseline parser actually used for the first experimental cycle, with **27 model features** after removing metadata and target. Newer parser branches already expose a richer feature space, including explicit counter-advantage and longer-horizon momentum features. This is important: the experimental results currently available mostly correspond to the earlier compact feature set, while the codebase is already evolving toward a second-generation dataset.

5.2 Combat score

A central domain-engineered variable is the `combat score`. Instead of using only raw unit counts, the parser builds a weighted military strength estimate based on:

- per-unit base power,
- number of alive units,
- matchup-dependent counter penalties,
- upgrade and technology multipliers.

In the earlier project material, this design was motivated as a simple approximation to combat effectiveness beyond raw supply. In the newer parser branch, the combat logic is paired with an explicit counter-advantage computation and longer-horizon volatility features.

5.3 Economic features

Economic state is summarized through worker counts, income-related statistics, and especially *Spending Quotient* (SQ), which tries to quantify how effectively a player converts resources and income into usable progress. In the prior project presentation, SQ is treated as a macroeconomic efficiency proxy and is one of the core inputs of all model families.

5.4 Scouting and composition features

The project also builds features that do not directly correspond to standard scoreboard metrics:

- scouting score derived from camera movement and enemy-base attention,
- army entropy as a proxy for compositional diversity,
- number of unit types as a coarse signal of strategic flexibility.

These variables are particularly important because they encode behavior and composition rather than only scale.

5.5 Dynamic features

The first parser generation already introduced local deltas such as changes in workers, combat score, and SQ. The newer parser branch extends this idea with multi-horizon momentum terms (15 s, 60 s, 120 s), growth ratios, and rolling volatility estimates. This is one of the clearest directions for the next experimental phase because the outcome prediction task is inherently temporal even if the current models consume independent rows.

6 Methodological Pivot: Leakage Control

The most important scientific lesson of the project is that **row-level random splitting is invalid** for this dataset. Adjacent snapshots from the same replay share the same players, strategy arc, map context, and eventual label. If one snapshot goes to train and a nearby snapshot from the same match goes to test, the model is rewarded for memorizing replay-specific context rather than learning general strategic patterns.

For this reason, all serious evaluation must use replay-based grouping:

- outer train/test split with `GroupShuffleSplit`,
- inner cross-validation with `GroupKFold` whenever model selection is performed,
- no use of test data for early stopping or hyperparameter selection.

Replay-aware validation settles near **61%**. This should be treated as the credible estimate for the preserved benchmark, whereas naive row-based validation is methodologically invalid because it leaks information across snapshots from the same replay.

7 Model Families and Expected Behavior

7.1 Random Forest

Random Forest is a strong baseline for this dataset because the inputs are already engineered tabular variables with nonlinear thresholds and interactions [1]. It does not require feature scaling, is robust to heterogeneous feature magnitudes, and can learn discrete rule-like partitions that fit many RTS situations.

7.2 XGBoost

XGBoost is attractive for the same reasons as Random Forest, but with sequential residual correction and stronger capacity to exploit subtle interactions [2]. The current implementation uses GPU acceleration with histogram-based tree construction and early stopping on a dedicated validation split.

7.3 MLP

The MLP branch is conceptually interesting because it tests whether a deep feedforward network can extract smooth nonlinear structure from the tabular representation, using a GPU-trained PyTorch implementation in the final branch [4]. Batch normalization, dropout, LeakyReLU, and quantile normalization are all sensible design choices for heterogeneous features. However, the deep model is also the most sensitive to methodological mistakes in validation and scaling.

7.4 Why trees may be favored here

The current working hypothesis is that tree-based models are naturally aligned with this problem because many RTS states behave like threshold systems: if one side has detector coverage, enough workers, enough tech, or enough army-quality advantage, the decision boundary can change abruptly.

A piecewise tree structure often captures these conditions more directly than a smooth multilayer perceptron trained on independent rows.

8 Experimental Protocol

8.1 Baseline data preparation

Across the current codebase, the canonical supervised setup is:

- drop `p1_wins`, `replay_id`, and `time_sec` from the predictors;
- remove NaN and infinite values;
- perform a replay-aware outer split for held-out testing;
- optionally create an inner replay-aware validation split for model selection and early stopping.

8.2 Random Forest protocol

The stronger Random Forest training script uses scikit-learn grouped model-selection utilities [3]:

- outer train/test split by replay,
- model selection with `GroupKFold`,
- final training on the full outer training partition,
- test-only evaluation on unseen replays.

To reduce grid-search cost, only full replays are subsampled for hyperparameter tuning rather than random rows, which is the correct strategy.

8.3 XGBoost protocol

The XGBoost branch is the cleanest current experimental implementation:

1. outer group-aware split into train/validation pool and test set,
2. inner group-aware split into train and validation,
3. GroupKFold grid search only on the inner training set,
4. early stopping on the validation split,
5. final evaluation only once on the untouched test split.

This is methodologically the strongest reference point in the current project files.

8.4 MLP protocol status

The deep model branch has now been rerun with a GPU-aware PyTorch implementation on the same **V3.1 fixed** dataset used for the aligned tree baselines. The current MLP protocol uses:

1. a replay-aware outer split into train/validation pool and test set,
2. a replay-aware inner split into train and validation,

3. candidate selection by validation log-loss across a small architecture set,
4. mixed-precision GPU training with early stopping on validation.

This does not provide exact methodological symmetry with the grouped GridSearchCV used by XGBoost and Random Forest, but it is far stronger than the earlier `sklearn` MLP branch and is sufficient to support a fair qualitative comparison between tree ensembles and a GPU-trained feedforward network.

9 Current Results Snapshot

9.1 Consolidated ablation status up to the current iteration

The project now contains a much more mature sequence of replay-aware XGBoost experiments than the first draft. The evaluation protocol has been cleaned so that the test set is never used for early stopping, and the dataset has evolved through multiple parser revisions. The current progression is not a collection of unrelated runs: it is effectively an ablation path over the dataset representation.

Table 1: Replay-aware XGBoost ablation progression across the current parser versions. All runs use grouped splitting by `replay_id`, grouped model selection, and a pure held-out test set.

Version	Main dataset / parser change	Acc.	Bal. Acc.	ROC-AUC	LogLoss
Baseline clean	Corrected train/validation/test protocol on the compact dataset	0.6063	0.6016	0.6509	0.6529
V2 dynamic	Added richer temporal features (60 s / 120 s trends, momentum, volatility, income-related dynamics)	0.6158	0.6118	0.6618	0.6477
V3 counter120	Added explicit counter features and longer-horizon income trends	0.6177	0.6135	0.6646	0.6461
V3.1 fixed	Corrected constants/counters semantics, repaired tech metadata, activated meaningful tech features	**0.6193**	**0.6154**	**0.6665**	**0.6451**
V3.2 combatfix	Refactored combat representation into raw and matchup-adjusted army power; semantically cleaner but nearly tied with V3.1	0.6195	0.6155	0.6660	0.6454

Two conclusions are already defensible. First, the largest gain comes from improving the *representation* rather than swapping the model family. Second, once the protocol is clean, the project sits in a narrow

but meaningful performance band around 0.62 accuracy, which is credible for tabular snapshots of a complex partially observed RTS game.

9.2 Current best XGBoost reference run

The ablation study now gives a clearer picture of the two leading parser branches. **V3.1 fixed** remains the safest main reference because it provides the best probabilistic quality, while **V3.2 combatfix** is the most accurate in raw classification and remains valuable as a semantically cleaner combat representation. Their reported grouped-validation and pure-test metrics are:

- **V3.1 fixed**: grouped CV 0.6066, best parameters learning rate = 0.05, max depth = 6, early stopping around 488 trees, test accuracy 0.6193, balanced accuracy 0.6154, ROC-AUC 0.6665, log loss 0.6451;
- **V3.2 combatfix**: grouped CV 0.6019, best parameters learning rate = 0.05, max depth = 6, early stopping around 601 trees, test accuracy 0.6195, balanced accuracy 0.6155, ROC-AUC 0.6660, log loss 0.6454.

This is effectively a statistical tie, but not a semantic tie. V3.1 fixed is easier to defend as the main baseline because its AUC and log-loss are slightly stronger and the branch directly reflects the correction of constants/counters semantics and repaired tech metadata. V3.2 combatfix remains useful as an ablation branch because it separates raw army power from matchup-adjusted power and clarifies the role of combat representation even without clearly surpassing V3.1 on all metrics.

9.3 Random Forest cross-check on the fixed dataset

A Random Forest experiment was rerun on the **V3.1 fixed** dataset under the same replay-aware train/validation/test protocol used for the final XGBoost line. For transparency, the revised package retains two RF artifact lines: an aligned benchmark summary for the headline cross-model table and a slightly different export-preserved rerun used in the bootstrap, calibration, and duration analyses. The resulting metrics are:

- grouped CV score on train folds: 0.6063,
- best parameters: 300 trees, `max_features=sqrt`, `min_samples_leaf=4`, `min_samples_split=10`, unrestricted max depth,
- held-out test accuracy: 0.6185,
- balanced accuracy: 0.6144,
- ROC-AUC: 0.6642,
- log loss: 0.6483.

This is extremely close to the XGBoost V3.1 result and supports a strong interpretation: after the parser and feature semantics are improved, much of the gain comes from the dataset itself rather than from a large gap between tree ensembles.

This cross-check matters because it suggests that the project is no longer dominated by algorithm choice alone. The current bottleneck is the information content of the snapshot representation, especially in the longer-duration and strategically richer phases of the game.

9.4 Final cross-model comparison on the frozen main dataset

The final model-family comparison was executed on the same **V3.1 fixed** dataset. XGBoost remains the best overall model, but the margin over Random Forest is extremely small. The GPU-trained PyTorch MLP is competitive in probabilistic ranking terms, yet remains clearly below the tree ensembles in classification accuracy.

Table 2: Final cross-model comparison on the frozen V3.1 fixed dataset. All results use replay-aware train/validation/test splitting.

Model	Training note	Acc.	Bal. Acc.	ROC-AUC	LogLoss
XGBoost	Grouped GridSearchCV, GPU histogram trees, early stopping on validation	**0.6193**	**0.6154**	**0.6665**	**0.6451**
Random Forest	Grouped GridSearchCV under the same replay-aware outer protocol	0.6180	0.6153	0.6643	0.6487
MLP PyTorch	GPU-trained feedforward network with validation-based candidate selection	0.6116	0.6124	0.6626	0.6483

The interpretation is now robust: the strongest gains came from parser correction and feature engineering rather than from moving toward more complex model classes. The small discrepancy between the Random Forest row in Table 2 and the RF values used later in Tests 1–4 is intentional: the package preserves both the aligned benchmark summary and a separate export-compatible RF rerun, and the latter is the one used whenever prediction-level comparisons are required. Tree ensembles remain the safest default on this tabular snapshot representation. The MLP result is still useful because it shows that a GPU-trained neural model can learn the same broad signal, but it does not provide evidence that feedforward deep learning outperforms strong tree baselines on the current feature space.

9.5 Paired replay-bootstrap significance test: XGBoost vs Random Forest

Because the final XGBoost and Random Forest results are extremely close, a paired replay-level bootstrap was run on the frozen **V3.1 fixed** test split. The bootstrap resampled `replay_id` with replacement and retained all snapshots belonging to each sampled replay, which is the correct statistical unit for this dataset because snapshots from the same match are not independent. The merged prediction set contains **178,699** test snapshots from **4,649** replays.

Table 3: Paired replay-bootstrap comparison between XGBoost and Random Forest on V3.1 fixed. Positive values favor XGBoost.

Difference metric	Mean diff.	95% CI	Interpretation
Accuracy ($\Delta_{\text{XGB}-\text{RF}}$)	+0.00083	[−0.00179, 0.00365]	CI includes zero; no strong 95% superiority claim for XGBoost on accuracy.
ROC-AUC ($\Delta_{\text{XGB}-\text{RF}}$)	+0.00231	[−0.00096, 0.00568]	CI includes zero; the AUC edge is consistent but not statistically decisive at 95%.
LogLoss ($\Delta_{\text{RF}-\text{XGB}}$)	+0.00320	[0.00089, 0.00545]	CI is strictly positive; XGBoost is statistically better in log-loss.

This result sharpens the final benchmark interpretation. XGBoost remains the nominally best model, but its advantage over Random Forest is *not* statistically robust on accuracy or ROC-AUC under a replay-aware paired bootstrap. The only clearly supported margin is on log-loss, meaning that the strongest evidence in favor of XGBoost lies in the quality of its probabilistic predictions rather than in a large difference in hard classification performance.

9.6 Calibration analysis of the final tree ensembles

The calibration test was then run directly on the frozen prediction exports from Test 1. Both models were evaluated on the same **178,699** test snapshots using Brier score, log-loss, ROC-AUC, Expected Calibration Error (ECE), and Maximum Calibration Error (MCE). The results are summarized below.

Table 4: Calibration comparison between XGBoost and Random Forest on the frozen V3.1 fixed test split. Lower Brier, log-loss, ECE, and MCE are better.

Model	Brier	LogLoss	ROC-AUC	ECE	MCE
XGBoost	**0.2273**	**0.6451**	**0.6665**	**0.0208**	**0.0294**
Random Forest	0.2286	0.6483	0.6642	0.0294	0.0791

The calibration evidence is unusually coherent. XGBoost is better than Random Forest on every probability-quality metric reported here: lower Brier score, lower log-loss, lower ECE, and much lower MCE. In other words, the replay-bootstrap and the calibration analysis tell the same story from two angles. Random Forest is almost tied with XGBoost as a classifier, but XGBoost is the safer final choice when calibrated probabilities matter.

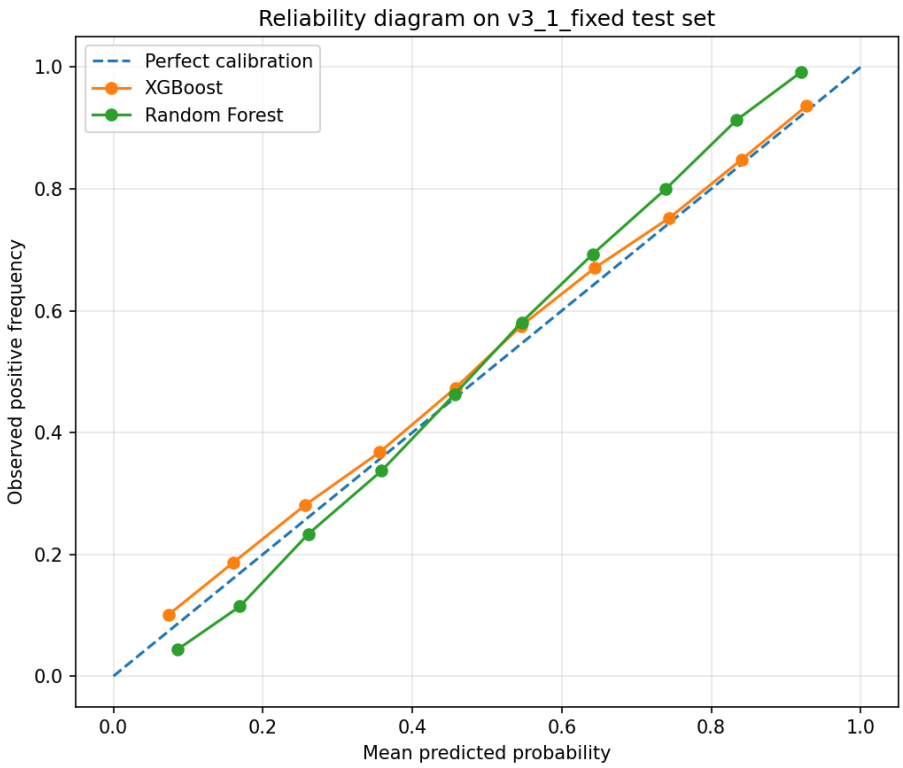

Figure 1: Reliability diagram for the frozen V3.1 fixed test split. XGBoost remains closer to the ideal calibration line across the probability bins, while Random Forest shows larger local deviations.

9.7 Calibration completion with the final MLP export

To close the probabilistic comparison across all final model families, the PyTorch MLP was re-exported on the same frozen **V3.1 fixed** test split and evaluated with the same 10-bin calibration protocol. The export run remained slightly below its earlier aligned benchmark but confirmed the same global ranking: XGBoost first, Random Forest second, and MLP third. Table 5 reports the final three-way calibration comparison.

Table 5: Final three-way calibration comparison on the frozen V3.1 fixed test split. Lower Brier, log-loss, ECE, and MCE are better.

Model	Brier	LogLoss	ROC-AUC	ECE	MCE
XGBoost	**0.2273**	**0.6451**	**0.6665**	**0.0208**	**0.0294**
Random Forest	0.2286	0.6483	0.6642	0.0294	0.0791
MLP PyTorch	0.2289	0.6488	0.6620	0.0328	0.0409

The additional MLP calibration run does not change the decision logic of the paper; it strengthens it. XGBoost remains best on every reported calibration-related metric, Random Forest remains a very strong second model with weaker probability quality but nearly tied discrimination, and the MLP lags behind both tree ensembles. Interestingly, MLP shows lower MCE than Random Forest, suggesting fewer extreme local deviations, but it is still worse on Brier score, log-loss, ROC-AUC, and ECE. The overall probabilistic ordering therefore remains *XGBoost* > *Random Forest* > *MLP*.

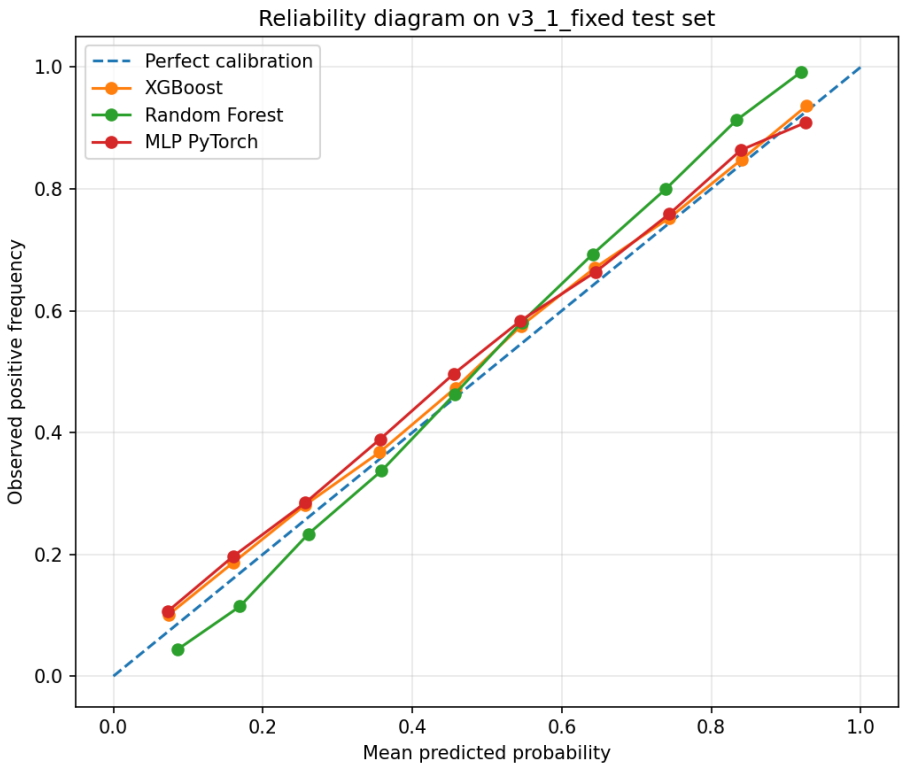

Figure 2: Final three-way reliability diagram on the frozen V3.1 fixed test split. XGBoost remains closest to ideal calibration overall, Random Forest is highly competitive but shows larger local deviations, and MLP PyTorch is systematically less calibrated than the two tree ensembles.

9.8 Frozen ablation and reproducibility table

Table 6: Frozen ablation summary under the uniform replay-aware XGBoost protocol. Every dataset contains 892,047 snapshots derived from 23,241 replays, split into 14,873 training replays, 3,719 validation replays, and 4,649 test replays.

Dataset	Feat.	CV Acc.	Test Acc.	Bal. Acc.	ROC-AUC	LogLoss	Best iter.
Baseline clean	27	0.5955	0.6063	0.6016	0.6509	0.6529	257
V2 dynamic	55	0.6007	0.6158	0.6118	0.6618	0.6477	367
V3.1 fixed	64	0.6066	0.6193	0.6154	**0.6665**	**0.6451**	488
V3.2 combatfix	73	**0.6019**	**0.6195**	**0.6155**	0.6660	0.6454	601

10 Feature Importance and Interpretation

The newer experiments sharpen the interpretation considerably. In the compact baseline, worker difference already dominated the feature ranking. After the parser revisions, the most stable and influential variables are now:

- `diff_income` and `diff_workers`,
- longer-horizon economic dynamics such as `diff_income_60s` and `diff_income_120s`,
- combat-related summaries such as `diff_combat_score` or, in the combatfix branch, `diff_raw_army_power`,

- scouting signals and number-of-unit-types features,
- repaired tech variables such as `tech_diff` and player-specific tech levels.

This evolution is scientifically important. It shows that the project has moved from a mostly macroeconomic predictor toward a more balanced representation that combines economy, temporal momentum, composition complexity, scouting, and technology timing. In particular, the jump from V2 to V3.1 confirms that the earlier constants/counters layer was not being exploited correctly: once its semantics were fixed, technology-related variables became genuinely useful.

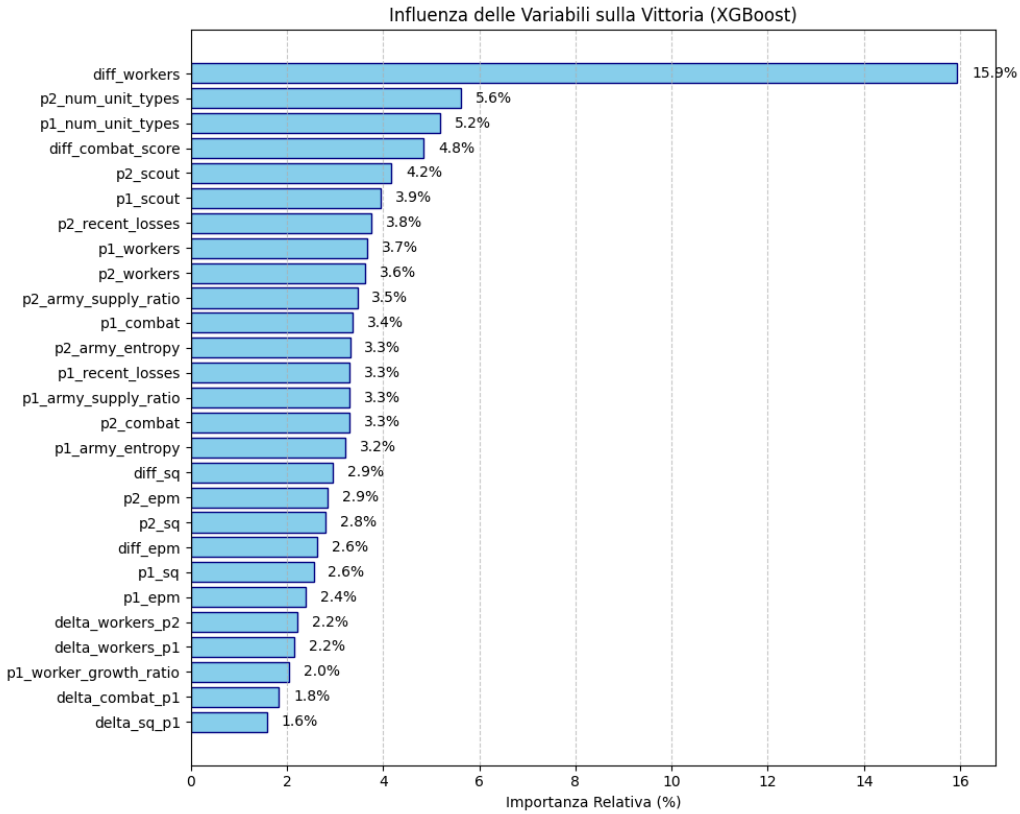

Figure 3: Earlier XGBoost feature-importance visualization from the compact feature set. It is retained as a baseline reference.

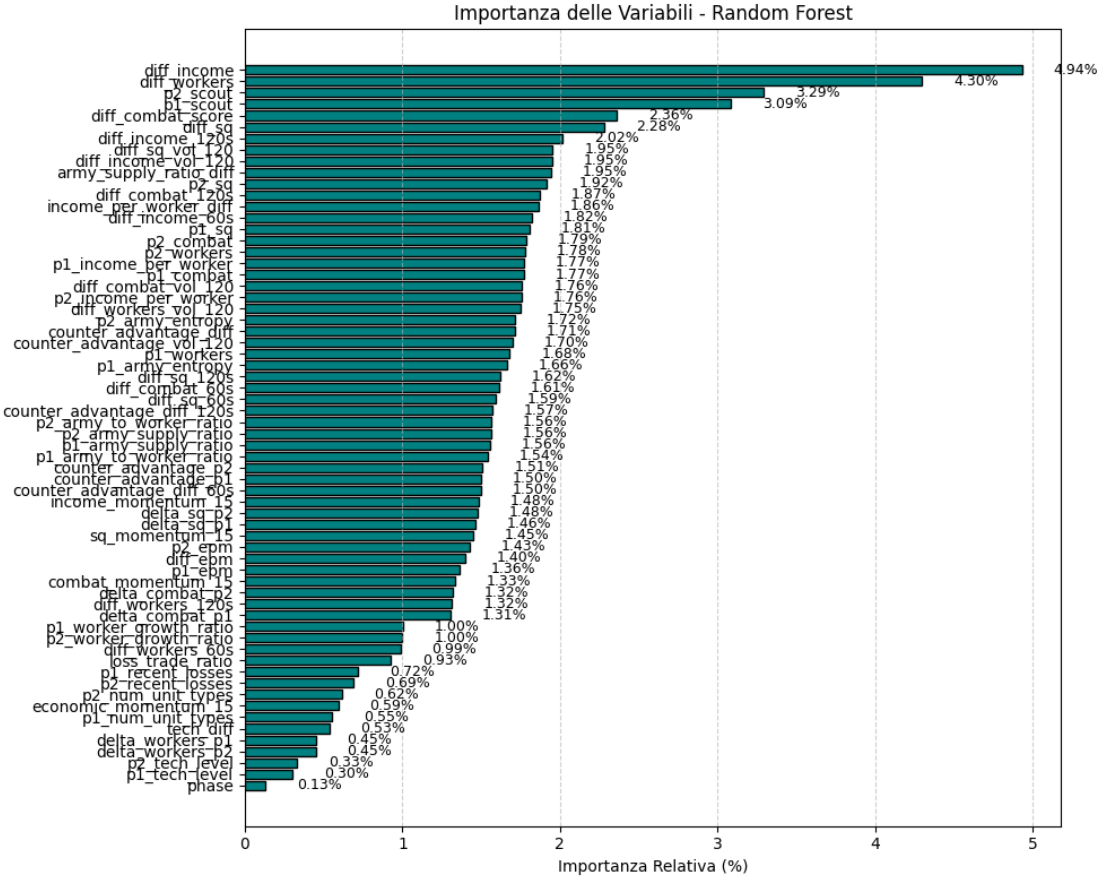

Figure 4: Random Forest feature importance on the V3.1 fixed dataset. The ranking is broadly consistent with the improved XGBoost runs: economy and momentum dominate, while scouting, combat, and counter/tech-related features contribute additional signal.

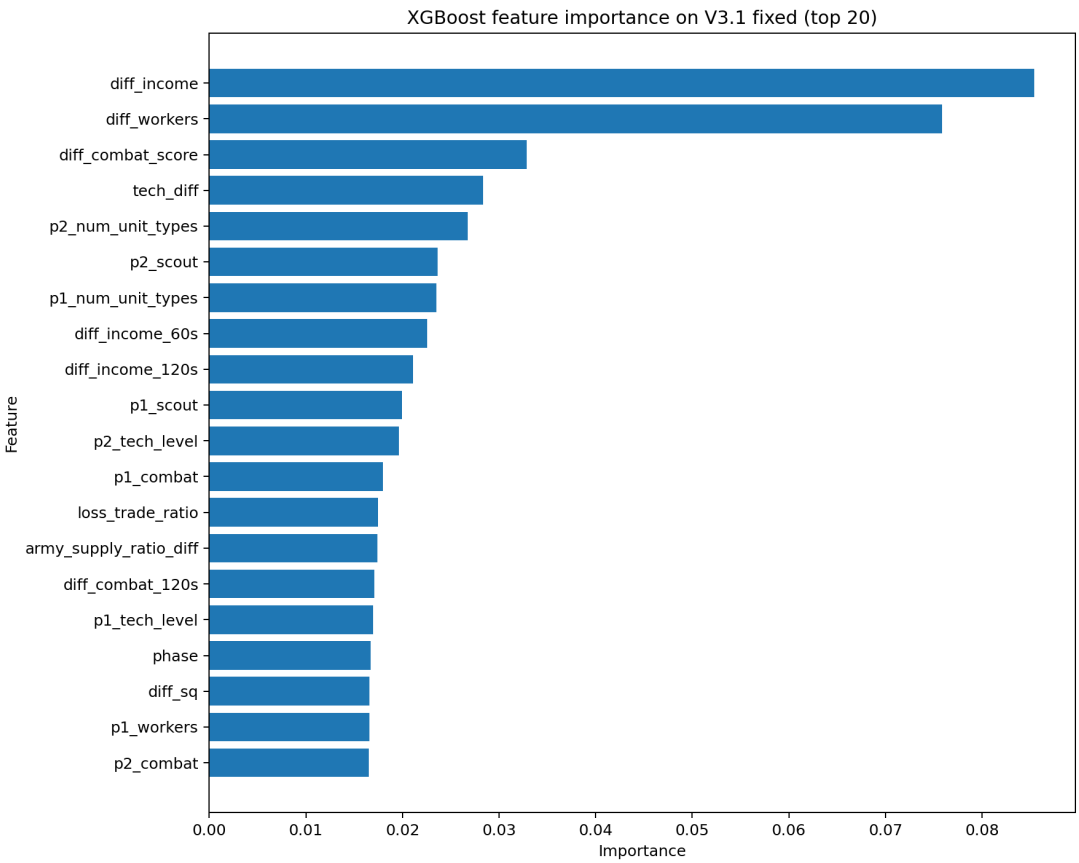

Figure 5: XGBoost feature importance on the frozen V3.1 fixed dataset. Economy and worker advantage remain dominant, while technology, combat, and army-complexity variables contribute the secondary signal that separates the richer parser branch from the earlier compact baseline.

11 Temporal Analysis by Match Duration

The unified duration comparison is now available for all three final models on the frozen **V3.1 fixed** test split. The comparison was built replay-aware: each snapshot inherits the total duration of its replay through the maximum observed `time_sec` for that replay, and all models are then evaluated on the same duration bins with matched `n_samples` and `n_replays`. The resulting comparison confirms that the late-game weakness is a shared property of the representation rather than a model-family-specific failure mode.

Table 7 summarizes the final duration-binned accuracy values. The strongest regime for all models remains the standard mid-game region around 5–10 minutes, where XGBoost is best overall. Random Forest is extremely competitive throughout and becomes slightly stronger than XGBoost in the longer bins. The PyTorch MLP remains consistently below both tree-based models in every populated bin.

Table 7: Final unified duration comparison on the frozen V3.1 fixed test split. All three models use the same replay-derived duration bins.

Duration bin	XGBoost	Random Forest	MLP PyTorch	$n_{samples}$	$n_{replays}$
0 min	0.5853	0.6153	0.5475	1,799	217
5 min	**0.6435**	0.6360	0.6300	38,911	1,705
10 min	**0.6285**	0.6268	0.6227	69,353	1,719
15 min	0.6119	**0.6144**	0.6075	41,627	688
20 min	0.5770	**0.5770**	0.5691	19,627	246
25 min	0.5684	**0.5820**	0.5622	7,382	74

The important conclusion is not that one model dominates every duration regime, but that *all* models lose accuracy in very long matches. The common decline after roughly 15–20 minutes suggests a representation ceiling: the current tabular state abstraction appears sufficient for economy, scouting, and medium-horizon momentum, but less sufficient for rich late-game composition, positioning, and hidden-information interactions.

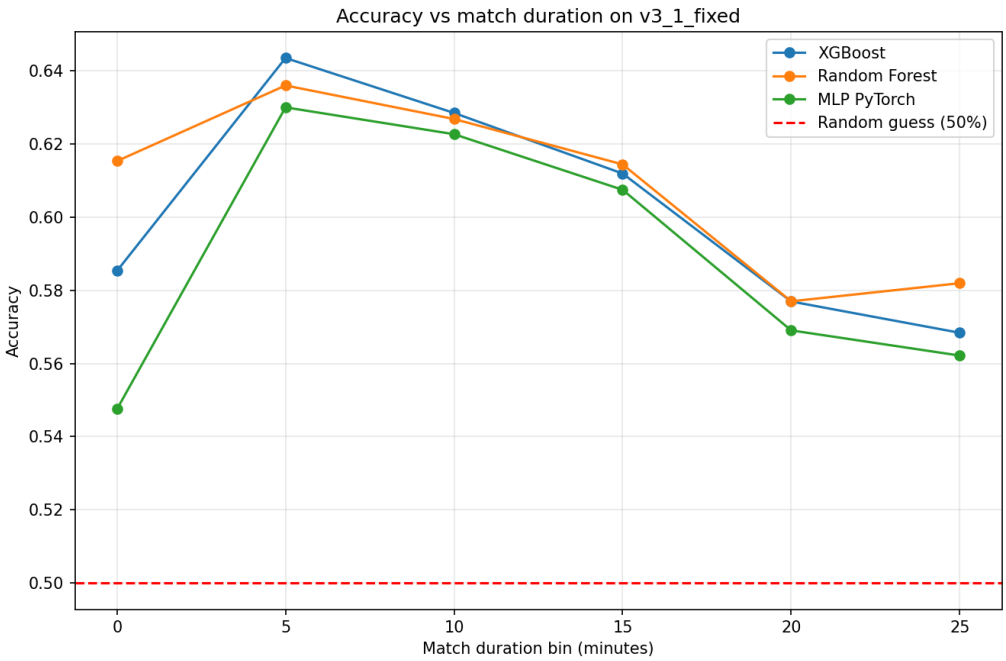

Figure 6: Final unified duration-based accuracy comparison on V3.1 fixed. XGBoost is strongest overall in the dominant 5–10 minute regime, Random Forest remains extremely close and can be slightly stronger in the longest bins, and MLP PyTorch stays consistently below both tree ensembles.

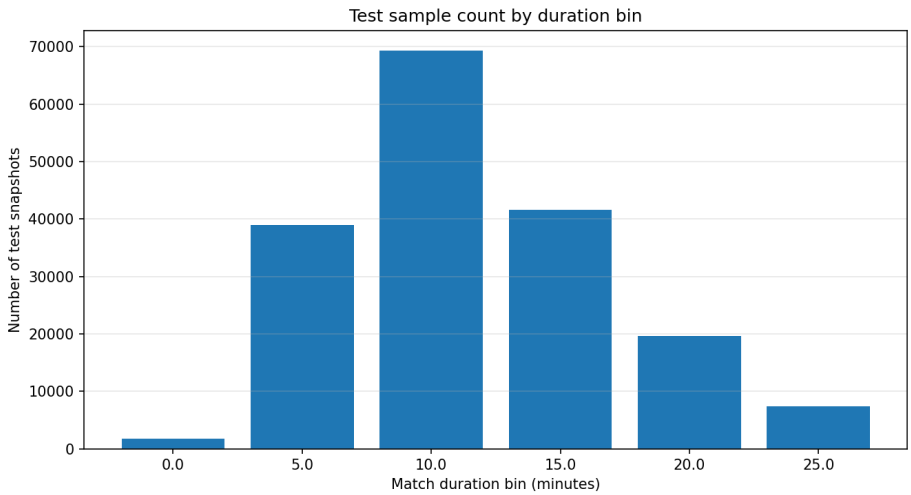

Figure 7: Test sample count by duration bin for the unified duration comparison. The longer-duration bins are noticeably smaller, which should be kept in mind when interpreting late-game differences.

12 Critical Discussion

12.1 Why the absolute accuracy is not extremely high

A 61% replay-aware accuracy may appear modest at first glance, but it is actually plausible for this representation. The dataset deliberately compresses a complex partially observed game into periodic scalar summaries. This loses:

- spatial positioning,
- engagement geometry,
- local terrain context,
- exact fog-of-war knowledge,
- fine-grained micro control.

Therefore, the model is predicting the winner from strategic summaries rather than from the full state of the game.

12.2 What the models are probably learning

The best interpretation is that the models learn *strategic momentum* rather than tactical certainty. They can detect when one player has accumulated enough economic, scouting, and army-structure advantage that the match becomes more likely to end in their favor. They are much weaker when the eventual result depends on hidden information, one decisive engagement, or execution details invisible to the feature space.

12.3 Why trees currently dominate

The current results support the claim that tree ensembles are the safest default on this dataset. This does not mean deep learning is useless here. It means that with independent tabular snapshots and

a relatively compact hand-engineered feature set, the inductive bias of trees may be more aligned with the problem than that of a feedforward neural network.

13 Threats to Validity

The current project should explicitly acknowledge the following validity threats:

1. **Parser-feature mismatch across branches.** The codebase already contains richer parser versions than the feature set used in the first completed experiments.
2. **Non-identical model-selection procedures across families.** XGBoost and Random Forest use grouped GridSearchCV, whereas the PyTorch MLP uses validation-based candidate selection over a smaller configuration set.
3. **Potential instability in late-game bins.** Very long games may be underrepresented, causing noisy duration trends.
4. **Representation ceiling.** Tabular snapshots may simply omit too much information for large gains without sequence or spatial modeling.

14 Conclusion

At its current stage, the project already supports a strong technical narrative. It demonstrates that match prediction in StarCraft II from tabular replay snapshots is feasible, but only if the pipeline is treated as a full machine learning system rather than a simple classifier benchmark. The main methodological contribution is the elimination of replay leakage through grouped evaluation. The main empirical result is that once validation is corrected and the parser is improved, the strongest baselines converge near 62% accuracy, with the dominant gains coming from better feature semantics and temporal representation rather than from changing the learning algorithm alone.

The final cross-model comparison sharpens that claim. XGBoost is the best overall model on the frozen V3.1 dataset, Random Forest is almost tied across the main metrics, and the GPU-trained PyTorch MLP remains consistently below the two tree ensembles. The paired replay-bootstrap refines the interpretation: XGBoost does not show a statistically strong 95% advantage over Random Forest on accuracy or ROC-AUC, but it does show a statistically supported advantage in log-loss. The calibration analysis then strengthens that conclusion further. In the two-model tree comparison, XGBoost is better calibrated than Random Forest, with lower Brier score, lower ECE, and much lower MCE on the frozen test split. The optional three-way calibration completion preserves the same ordering: XGBoost remains best on all reported probability-quality metrics, Random Forest remains second, and MLP is third overall despite having a lower MCE than Random Forest. This is a good outcome, not a disappointing one. It means the project has already moved past naive benchmark inflation and reached the stage where progress must come from better representation, cleaner experiments, and more explicit temporal and counter-based modeling. In other words, the main reason to prefer XGBoost is no longer a large gain in raw accuracy, but the combination of near-best discrimination and clearly better probabilistic quality.

15 Conceptual Extension: A POMDP-like View of Replay Snapshots

Although the present study is framed as a supervised learning problem over tabular replay snapshots, the underlying *StarCraft II* process is naturally partially observable. The true strategic state of

a match at time t is richer than the compact feature vector exposed to the predictive models. It includes latent or only partially visible factors such as hidden tech choices, spatial pressure, future production intent, local tactical geometry, and information asymmetry. For this reason, the current pipeline also admits a useful *POMDP-like* conceptual interpretation.

Let $s_t \in S$ denote the latent strategic state of the match at time t, and let $o_t \in O$ denote the observed replay snapshot extracted by the parser and feature engineering pipeline. Under this view, the dataset does not provide direct access to the full strategic state. Instead, it provides a compact and engineered observation of that state, built from economy, army composition, scouting, recent losses, and short-horizon temporal deltas. A high-level action space A can then be introduced conceptually, where actions correspond not to low-level game commands but to abstract strategic choices such as economic investment, worker growth, military production, defensive stabilization, offensive pressure, or additional scouting.

In this interpretation, match evolution may be described by a transition model $T(s_{t+1} \mid s_t, a_t)$ and an observation model $Z(o_{t+1} \mid s_{t+1}, a_t)$, where the quality of the future observation is influenced by partial information and scouting quality. Historical replay trajectories can then be viewed as empirical evidence for estimating a belief-like representation over latent strategic situations. The current project does not explicitly solve a POMDP, perform belief updates, or optimize a sequential policy. However, the replay dataset can still be interpreted as a collection of observation sequences that act as proxies for hidden strategic states and that could support future work on latent-state estimation, action valuation, or policy recommendation under partial observability.

This perspective should be understood as a conceptual extension rather than a redefinition of the present contribution. The main result of the paper remains a rigorous replay-aware machine learning pipeline for match outcome prediction. Nevertheless, the POMDP-like view is useful because it clarifies why static tabular snapshots already contain meaningful strategic signal while also explaining why future progress may require richer temporal modeling, belief-like state tracking, and more explicit reasoning over strategic actions. A separate supplementary note develops this interpretation in a more compact formal style.

A Appendix A: Implementation Snapshot

The implementation analyzed in this study contains the following main components:

- a baseline parser and bridge used for the earlier compact dataset,
- updated parser and bridge branches with explicit counter, momentum, volatility, tech-fix, and combat-refactor features,
- an XGBoost pipeline with proper outer split, inner split, grouped model selection, and early stopping,
- a Random Forest pipeline with grouped model selection,
- a GPU-aware PyTorch MLP pipeline using replay-aware train/validation/test splitting and validation-based candidate selection.

B Appendix B: Supplementary Note on the POMDP-like Interpretation

A separate supplementary note expands the conceptual formulation introduced near the end of the main paper. Its purpose is to formalize the distinction between latent strategic state, replay-derived

observation, high-level strategic action, observation quality, and belief-like estimation from historical replay trajectories.

This material should be read as a conceptual extension and as a possible future theoretical direction. It does not change the empirical scope of the present study, which remains centered on replay-aware supervised outcome prediction.

References

[1] Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016.

[3] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Èdouard Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.